Hyper-V VM Creation

PowerShell guide | External & Internal network modes | Mac keyboard support

This guide covers creating an Ubuntu Server VM on Hyper-V using PowerShell, with two network modes: External (local server with router access) and Internal (cloud-hosted server with no router access, using Windows NAT). Also covers Mac keyboard support via Enhanced Session Mode and xRDP, and fixing eth0 autoconfiguration failures during Ubuntu install.

Network Mode Overview

Set $networkMode in the script config section before running:

Mode	When to use	VM internet access	Mac connects via
External	Local server with router/DHCP	Direct via LAN	VM IP:3389
Internal	Cloud server, no router access	NAT through Windows host	Server public IP:3390

In Internal mode the script automatically: creates the Internal switch, assigns the host gateway IP (192.168.100.1) to the vEthernet adapter, creates a Windows NAT, and adds a port-forward rule so that port 3390 on the Windows host routes to port 3389 on the VM.

Prerequisites

Verify Hyper-V is enabled. If not, enable it and reboot:

```
Get-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V
# Enable if needed:
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All
```

Phase 1 - Config Block (edit before running)

Open Create-UbuntuVM-v2.ps1 and set these variables at the top of the CONFIG section:

```
$networkMode = "Internal"     # "External" or "Internal"

# VM settings
$vmName    = "UbuntuServer"
$isoPath   = "C:\ISOs\ubuntu-24.04-live-server-amd64.iso"
$vhdSizeGB = 40GB
$ramGB     = 2GB
$cpuCount  = 2

# External mode only
$nicName   = "Ethernet"       # run Get-NetAdapter to find your adapter name

# Internal / NAT mode only
$hostGatewayIP  = "192.168.100.1"   # host IP on the internal network
$vmStaticIP     = "192.168.100.2"   # static IP to assign to Ubuntu
$hostForwardPort = 3390             # port on host that forwards to VM RDP
```

Phase 2 (Internal mode) - Switch, NAT, and Port Forward

When $networkMode is 'Internal' the script runs these steps automatically:

```
# 1. Create Internal virtual switch
New-VMSwitch -Name "InternalSwitch" -SwitchType Internal

# 2. Assign host gateway IP to the vEthernet adapter
```

```
$vEthernet = Get-NetAdapter | Where-Object { $_.Name -like "*InternalSwitch*" }
New-NetIPAddress -IPAddress 192.168.100.1 -PrefixLength 24 -InterfaceIndex $vEthernet.ifIndex

# 3. Create NAT so the VM can reach the internet through the host
New-NetNat -Name "VMNat" -InternalIPInterfaceAddressPrefix 192.168.100.0/24

# 4. Port forward: host port 3390 -> VM port 3389 (xRDP)
netsh interface portproxy add v4tov4 `
    listenport=3390 listenaddress=0.0.0.0 `
    connectport=3389 connectaddress=192.168.100.2

# 5. Open the forwarded port in Windows Firewall
New-NetFirewallRule -DisplayName "Hyper-V VM xRDP forward (3390)" `
    -Direction Inbound -Protocol TCP -LocalPort 3390 -Action Allow
```

Note: The NAT gives the VM outbound internet access (apt-get, git, etc.) through the Windows host's network connection. The port forward lets your Mac reach the VM's xRDP without the VM having a public IP.

Phase 2 (External mode) - Virtual Switch Validation

When $networkMode is 'External' the script validates the switch and adapter:

```
# Checks switch exists and is type External (recreates if Internal/Private)
# Checks physical adapter exists and is Up
# Prints host IP and gateway for reference if you need a static IP

# Manual creation if running steps individually:
New-VMSwitch -Name "ExternalSwitch" -NetAdapterName "Ethernet" -AllowManagementOS $true
```

Switch Type	Result in Ubuntu installer
External (correct)	eth0 gets DHCP address automatically
Internal or Private	eth0 autoconfiguration fails - no DHCP

Phase 3 - Create VHD and VM

```
New-Item -ItemType Directory -Path "C:\VMs\UbuntuServer" -Force
New-VHD  -Path "C:\VMs\UbuntuServer\UbuntuServer.vhdx" -SizeBytes 40GB -Dynamic

New-VM -Name "UbuntuServer" -Generation 2 -MemoryStartupBytes 2GB `
       -VHDPath "C:\VMs\UbuntuServer\UbuntuServer.vhdx" `
       -SwitchName $activeSwitch -Path "C:\VMs"

Set-VMProcessor -VMName "UbuntuServer" -Count 2
Set-VMMemory    -VMName "UbuntuServer" -DynamicMemoryEnabled $true `
                -MinimumBytes 512MB -MaximumBytes 4GB -StartupBytes 2GB
Set-VMFirmware  -VMName "UbuntuServer" -EnableSecureBoot Off
Enable-VMIntegrationService -VMName "UbuntuServer" -Name "Guest Service Interface"
Set-VM -VMName "UbuntuServer" -AutomaticStartAction Start -AutomaticStartDelay 30
Set-VM -VMName "UbuntuServer" -AutomaticStopAction ShutDown
```

Phase 4 - Ubuntu Installer Network Configuration

When the Ubuntu installer reaches the Network screen, configure eth0 manually based on the mode you are using:
Internal mode - always use a static IP (no DHCP on internal switch):

Field	Value	Notes
Subnet	192.168.100.0/24	Fixed - matches the NAT subnet in the PS1 script

Address	192.168.100.2	Static IP assigned to this VM
Gateway	192.168.100.1	Windows host vEthernet adapter IP (created by script)
Name servers	8.8.8.8, 8.8.4.4	Google DNS - traffic routes out via Windows NAT
Search domains	(leave blank)	Not needed unless you have a private DNS domain

Note: If you add a second VM later, assign it 192.168.100.3 and keep the same Gateway, Name servers, and Search domains values.

External mode - if eth0 autoconfiguration fails, set a static IP using the values printed by the script (host subnet and gateway):

Field	Value	Notes
Subnet	<host-subnet>	Shown by script at startup (e.g. 192.168.1.0/24)
Address	<free IP in LAN range>	Pick any unused IP in the same subnet
Gateway	<default gateway>	Shown by script at startup (your router IP)
Name servers	8.8.8.8, 8.8.4.4	Or use your router IP as DNS
Search domains	(leave blank)	Not needed

Persist static IP after install using Netplan (run inside Ubuntu):

```
ip link show   # find your interface name (eth0, ens3, ens18, etc.)

sudo nano /etc/netplan/00-installer-config.yaml

# Internal mode content:
network:
  version: 2
  ethernets:
    ens3:                          # replace with your interface name
      dhcp4: false
      addresses: [192.168.100.2/24]
      routes:
        - to: default
          via: 192.168.100.1
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]

sudo netplan apply
ping 8.8.8.8   # verify internet access through NAT
```

! Netplan YAML requires spaces for indentation, never tabs. Wrong indentation causes netplan apply to fail.

Phase 5 - Enhanced Session Mode (Mac Keyboard Fix)

The script enables Enhanced Session Mode automatically. This switches the Hyper-V console to use RDP internally, which translates Mac keyboard layouts correctly:

```
Set-VMHost -EnableEnhancedSessionMode $true
Set-VM -VMName "UbuntuServer" -EnhancedSessionTransportType HvSocket
```

Without Enhanced Session	With Enhanced Session
Basic video feed	RDP protocol inside Hyper-V
Mac keys not translated	Mac keyboard fully mapped
No clipboard sharing	Clipboard works between Mac and VM

Phase 6 - Install xRDP on Ubuntu

SSH into the VM and install xRDP to enable Mac Remote Desktop access:

```
sudo apt-get update
sudo apt-get install -y xrdp
sudo systemctl enable xrdp
sudo systemctl start xrdp
sudo ufw allow 3389/tcp
echo 'setxkbmap -layout us' >> ~/.bashrc

# Print IP (for External mode)
ip addr show | grep 'inet ' | grep -v 127.0.0.1
```

Phase 7 - Connect from Mac (Microsoft Remote Desktop)

Install 'Microsoft Remote Desktop' from the Mac App Store (free), then add a connection:

Setting	External mode	Internal mode (cloud)
PC Name / IP	VM IP (e.g. 192.168.1.50)	Cloud server public IP
Port	3389	3390 (forwarded to VM)
Username	Ubuntu username	Ubuntu username
Password	Ubuntu password	Ubuntu password

Mac Key	Sent to Ubuntu
Option + key	AltGr / special chars (|, @, #, etc.)
Command	Super / Meta key
Control	Ctrl
Fn + Delete	Forward delete

Phase 8 - Post-Install Cleanup

```
# Eject ISO
Set-VMDvdDrive -VMName "UbuntuServer" -Path $null

# Check port forward is active (Internal mode)
netsh interface portproxy show v4tov4

# Check NAT is active (Internal mode)
Get-NetNat -Name "VMNat"

# Verify Enhanced Session
Get-VM -VMName "UbuntuServer" | Select-Object Name, EnhancedSessionTransportType
```

Managing Port Forwards (Internal mode)

Useful commands for managing the port forward and NAT:

```
# View all port forwards
netsh interface portproxy show v4tov4

# Remove a port forward
netsh interface portproxy delete v4tov4 listenport=3390 listenaddress=0.0.0.0

# Add a second VM forward on a different port (e.g. second VM on 192.168.100.3)
netsh interface portproxy add v4tov4 `
    listenport=3391 listenaddress=0.0.0.0 `
    connectport=3389 connectaddress=192.168.100.3
```

```
# Remove NAT entirely
Remove-NetNat -Name "VMNat" -Confirm:$false
```

Useful Day-to-Day Commands

```
Get-VM
Start-VM   -Name "UbuntuServer"
Stop-VM    -Name "UbuntuServer" -Force
Restart-VM -Name "UbuntuServer" -Force

Checkpoint-VM -Name "UbuntuServer" -SnapshotName "Before Docker Install"
Restore-VMCheckpoint -Name "UbuntuServer" -VMCheckpointName "Before Docker Install" -Confirm:$false

Get-VMNetworkAdapter -VMName "UbuntuServer" | Select-Object -ExpandProperty IPAddresses

Remove-VM   -Name "UbuntuServer" -Force
Remove-Item -Recurse -Force "C:\VMs\UbuntuServer"
```

Dock Tools - Installation Inside the VM

Run these commands inside the VM after Ubuntu/Debian is installed and has network access.

Step 1 - Install Docker:

```
sudo apt-get update && sudo apt-get upgrade -y
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
newgrp docker
```

Step 2 - Clone the project:

```
# Create /opt/dock-tools and take ownership in one step
sudo mkdir -p /opt/dock-tools && sudo chown $USER:$USER /opt/dock-tools
git clone https://github.com/DSerruya/dock_tools.git /opt/dock-tools
cd /opt/dock-tools
```

! Permission denied on /opt? Your user may not have sudo rights. Fix as root:

```
su -                                          # switch to root
mkdir -p /opt/dock-tools
chown <your-username>:<your-username> /opt/dock-tools
exit                                          # back to normal user
git clone https://github.com/DSerruya/dock_tools.git /opt/dock-tools
```

Alternative - clone into home directory (no root needed):

```
git clone https://github.com/DSerruya/dock_tools.git ~/dock-tools
cd ~/dock-tools
```

Note: If you clone into ~/dock-tools, update HOST_SCRIPTS_DATA_PATH in .env to: /home/<your-username>/dock-tools/scripts-data

Step 3 - Configure the environment file:

```
cp .env.example .env
nano .env
```

Variable	Value	Notes
WEBHOOK_SECRET	openssl rand -hex 32	Run that command and paste the output
HOST_SCRIPTS_DATA_PATH	/opt/dock-tools/scripts-data	Change to ~/dock-tools/scripts-data if cloned in home
DEFAULT_TIMEZONE	UTC	Or your local timezone
MANAGER_PORT	80	HTTP port exposed by nginx

UI_USERNAME	admin	Web UI login username
UI_PASSWORD	<strong-password>	Web UI login password

Step 4 - Build and start:

```
cd /opt/dock-tools   # or ~/dock-tools if cloned in home
docker compose up -d --build

# Verify both services are running
docker compose ps

# Tail logs to confirm healthy startup
docker compose logs -f
```

Expected output from docker compose ps:

Name	Image	Status	Ports
script-manager	dock-tools-manager	Up	3000/tcp (internal)
script-nginx	nginx:alpine	Up	0.0.0.0:8484->80/tcp

Step 5 - Access the web UI:

Mode	URL	Notes
Internal (cloud)	http://192.168.100.2:8484	Default port 8484 - check MANAGER_PORT in .env
External (LAN)	http://<vm-ip>:8484	Use the port shown in docker compose ps

! Seeing 'Welcome to nginx' instead of Dock Tools? You are hitting the wrong port. Check your .env for MANAGER_PORT and add it to the URL. Run: grep MANAGER_PORT ~/dock-tools/.env

Credentials:

Field	Value
UI Username	admin
UI Password	admin123
Webhook Secret	ac15923a20c120a97dea6c7024fc79db0e86e3cef01b49a872efac429089343d

Note: These credentials were retrieved from the Rancher/Kubernetes dock-tools-secret. Keep this document secure and consider changing the UI password on a new deployment.

Step 6 - Auto-start on VM reboot:

```
sudo systemctl enable docker

sudo tee /etc/systemd/system/dock-tools.service > /dev/null <<EOF
[Unit]
Description=Dock Tools
After=docker.service
Requires=docker.service

[Service]
WorkingDirectory=/opt/dock-tools
ExecStart=/usr/bin/docker compose up
ExecStop=/usr/bin/docker compose down
Restart=always
User=$USER

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
```

```
sudo systemctl enable dock-tools
```

Guided Installer (install-guided.sh)

The guided installer handles the full setup interactively with no extra commands. Copy install-guided.sh to the VM, make it executable, and run it. It asks all required values, validates every path, deploys the stack, and runs a demo script to confirm the full pipeline is working.

```
chmod +x install-guided.sh
./install-guided.sh
```

The installer walks through the following steps:

Step	What happens	Mode
1 - Deployment target	Choose Docker Compose (Ubuntu/Debian) or Rancher/K8s	Both
2 - Dependencies	Installs Docker, git, curl; handles docker group session fix	Compose
2 - Prerequisites	Checks kubectl, docker, cluster context	Rancher
3 - Install directory	Clones repo or pulls latest; creates scripts-data dir	Both
4 - Configuration	Asks all .env values interactively (see table below)	Both
5 - Build & deploy	docker compose up --build OR kubectl apply -k k8s/	Both
6 - Validate services	Checks containers are up, web UI responds, mount works	Both
7 - Demo script	Creates local Python heartbeat, adds via API, checks logs	Compose
8 - GitHub test	Optional: clones a public repo to test git connectivity	Compose
9 - Auto-start	Optional: creates systemd service for boot auto-start	Compose

All .env values collected during Step 4:

Variable	Description	Default
Webhook secret	HMAC-SHA256 key for GitHub webhooks	Auto-generated (openssl rand -hex 32)
Timezone	Default timezone for cron schedules	Auto-detected from system
HTTP port	Host port mapped to nginx container	8484 (avoids conflicts with system port 80)
HTTPS port	TLS port (only if TLS enabled)	8443 (avoids conflicts with system port 443)
UI username	Web UI login username	admin
UI password	Web UI login password (required)	You choose - cannot be empty
TLS enabled?	Generate self-signed cert, use HTTPS	No (optional)
scripts-data path	HOST_SCRIPTS_DATA_PATH on host	~/dock-tools/scripts-data (auto-set)

Note: Ports 80 and 443 require root privileges on Linux and are often occupied by other services on cloud servers. The installer defaults to 8484/8443 which work without sudo.

Demo script (Step 7) - validates the local pipeline end-to-end:

Check	How it works
Volume mount correct	Creates repo in scripts-data, manager clones it via file:// URL
Container creation	Dock Tools API starts a Python container
Script execution	Python heartbeat prints timestamped output every 5 seconds
Log streaming	Installer fetches logs via API and confirms output is present
scripts-data populated	Checks that <scripts-data>/demo-heartbeat/repo exists on host

Note: demo-heartbeat (Python) stays running after install as a permanent pipeline health indicator.

GitHub PAT and webhook test (Step 8) - validates GitHub connectivity:

Test	What is checked	Cleaned up?
PAT validation	Calls api.github.com/user with the token, confirms authenticated user	N/A
Git clone	Direct clone of dock_tools_test repo (with PAT or public)	Yes
Webhook delivery	Signed HMAC-SHA256 POST to /webhook/git-test, signature validate	Yes
Log output	Fetches logs to confirm stdbuf -o0 stdout fix is working	Yes

Test repo used by the installer:

Field	Value
URL	https://github.com/DSerruya/dock_tools_test
Visibility	Public - no PAT required for clone
Script	main.rb - Ruby heartbeat with $stdout.sync = true
Entry point	stdbuf -o0 ruby main.rb
Purpose	Validates git clone, webhook, and log streaming in one test

```
# main.rb content (dock_tools_test repo):
$stdout.sync = true
$stderr.sync = true

puts "=" * 55
puts "  Dock Tools - Git & Log Test"
puts "  Started: #{Time.now}"
puts "=" * 55
count = 0
loop do
  count += 1
  puts "[#{Time.now.strftime("%H:%M:%S")}] heartbeat ##{count} - pipeline OK"
  sleep 5
end
```

What remains in Dock Tools UI after a full install:

Script	Language	Remains?	Purpose
demo-heartbeat	Python	Yes - permanent	Local pipeline health check
git-test	Ruby	No - cleaned up	GitHub/webhook test only

! If the demo script fails with 'package.json not found', HOST_SCRIPTS_DATA_PATH is wrong. The installer sets this automatically - if you cloned to ~/dock-tools the value is ~/dock-tools/scripts-data. Verify with: grep HOST_SCRIPTS_DATA_PATH ~/dock-tools/.env

! Docker group permission denied? The installer detects this automatically and uses 'sudo docker' for the session. After install completes, reconnect your SSH session so future docker commands work without sudo. Do NOT run 'newgrp docker' on a headless server - it causes 'Cannot open display' errors.

docker-compose.yml key fixes applied (auto-applied via git pull):

Fix	What was wrong	What was fixed
UI_PASSWORD not pas	Manager started without auth - UI was publicly op	Added UI_USERNAME and UI_PASSWORD to manager e
Wrong default ports	Port 80/443 default caused conflicts on cloud serv	Changed fallback defaults to 8484/8443 in docker-compos
Obsolete version field	version: '3.8' caused a warning on every compose	Removed the version attribute

After pulling these fixes, restart the stack:

```
cd ~/dock-tools
git pull
sudo docker compose down
```

```
sudo docker compose up -d

# Confirm UI_PASSWORD reached the manager
sudo docker exec script-manager env | grep UI_PASSWORD
```

Rancher/K8s path (Step 6) additional steps:

```
# Installer runs these automatically:
kubectl apply -f k8s/namespace.yaml
kubectl create secret generic dock-tools-secret --namespace dock-tools ...
kubectl apply -k k8s/
kubectl rollout status deployment/dock-tools-manager -n dock-tools --timeout=120s
kubectl rollout status deployment/dock-tools-nginx   -n dock-tools --timeout=120s
```

UI Features - HowTo? and Admin Quick-Add

Two UI additions make adding and configuring scripts faster:

Feature	Where	What it does
? button	Top-right header, next to user badge	Opens HowTo modal with per-language field examples and cop
Heartbeat quick-add card	Admin tab, top of page	Pre-fills Add Script modal with dock_tools_test Ruby heartbeat v

HowTo? modal - field examples per language:

Language	Entry Point	Build Command	Key note
Ruby	stdbuf -o0 ruby main.rb	bundle install	stdout buffering fix included
Python	python -u main.py	pip install -r requirements.txt	-u flag = unbuffered output
Node.js	node index.js	npm install	No buffering issues
TypeScript	npx ts-node index.ts	npm install	Or: npm run build + node dist/index.js

Note: Every field in the HowTo modal has a copy button. Click it to copy the value directly into the clipboard.

Heartbeat quick-add (Admin tab) pre-fills the Add Script modal with:

Field	Pre-filled value
Name	heartbeat-test
Language	Ruby
Repo URL	https://github.com/DSerruya/dock_tools_test.git
Branch	main
Entry Point	stdbuf -o0 ruby main.rb
Build Command	(empty - no dependencies)

Note: All fields are editable before submitting. Add a GitHub Token if you fork the repo as private.

To pick up UI changes after a git pull, restart the stack:

```
cd ~/dock-tools
git pull
sudo docker compose restart
```

Dock Tools Troubleshooting

Symptom	Cause	Fix
Welcome to nginx page	Accessing wrong port - MANAGER_P	Add port to URL: http://<ip>:8484 (check .env for MANAGER_PORT
No UI_PASSWORD set (in l	UI_PASSWORD missing from docke	git pull to get the fix, then docker compose down && up -d
package.json not found (scr	HOST_SCRIPTS_DATA_PATH in .e	grep HOST_SCRIPTS_DATA_PATH .env and compare to actual pa

docker: permission denied	User not in docker group or session r	Use sudo docker, then reconnect SSH (do NOT run newgrp on head
eth0 autoconfiguration failed	No DHCP on internal switch, or switch	Set static IP manually in installer (see network config section)
Containers exited immediate	Port conflict, low disk, or bad .env val	Run: sudo docker compose logs --tail=40 to see the exact error

After pulling fixes, always restart the stack to apply changes:

```
cd ~/dock-tools
git pull
sudo docker compose down
sudo docker compose up -d

# Verify containers are running
sudo docker compose ps

# Check manager received correct env vars
sudo docker exec script-manager env | grep -E 'UI_|HOST_|WEBHOOK'
```

Script Logging - Limits and Real-time Output

Dock Tools captures stdout and stderr from script containers via Docker logs. Understanding the limits and buffering behaviour prevents missing output in the UI.

Scenario	Line limit	Real-time?
Logs tab - snapshot on open	Last 500 lines	Yes - then streams live
/api/scripts/:name/logs	Last 200 lines (default)	No - snapshot only
/api/scripts/:name/logs?tail=N	N lines (your choice)	No - snapshot only
Live stream while running	No limit	Yes - SSE real-time
Log file on disk (scripts-data)	No limit - kept forever	N/A - persistent file

Ruby stdout buffering - the most common reason logs appear empty or delayed:
Ruby buffers stdout by default when running non-interactively inside a container. Output is held in an 8 KB internal buffer and only flushed when it fills up or the script exits - so 'puts' lines may not appear for minutes in the live log view.

Fix	How to apply	Changes script?
$stdout.sync = true	Add at top of Ruby script	Yes
stdbuf -o0 ruby main.rb	Set as the entry point in Dock Tools UI	No

Option 1 - add to the Ruby script (fix travels with the code):

```
$stdout.sync = true
$stderr.sync = true

puts 'Script started'
loop do
  puts "#{Time.now} - doing work..."
  sleep 5
end
```

Option 2 - set in the Dock Tools entry point field (no code change needed):

```
stdbuf -o0 ruby main.rb
```

Note: Use stdbuf when the script is from a GitHub repo you do not control or when you cannot edit the source. It forces unbuffered output at the OS level and works for all output including stdout and stderr.

Language	Buffering fix
Ruby	$stdout.sync = true OR stdbuf -o0 ruby main.rb
Python	sys.stdout.flush() after each print OR python -u main.py

Node.js	No buffering issue - console.log() is unbuffered by default

Override the default 200-line API limit to get more log history:

```
# Get last 5000 lines via API
curl -u admin:password http://192.168.100.2:8484/api/scripts/my-script/logs?tail=5000
```

Network Diagnosis Checklist

Check	Command	Where
Interface name	ip link show	Ubuntu
Current IP	ip addr show	Ubuntu
Internet via NAT	ping 8.8.8.8	Ubuntu
NAT active	Get-NetNat	Windows (PS)
Port forward active	netsh interface portproxy show v4tov4	Windows (PS)
Switch type	Get-VMSwitch | Select Name,SwitchType	Windows (PS)
Adapter status	Get-NetAdapter | Select Name,Status	Windows (PS)